



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
Технології розробки програмного забезпечення
«ШАБЛОНИ «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF
RESPONSIBILITY», «PROTOTYPE»»

Виконав:
студент групи ІА-24
Іскандаров Е. Е.

Перевірив:
Мягкий М. Ю.

Зміст

Короткі теоретичні відомості.....	2
Реалізація шаблону проектування для майбутньої системи.....	3
Зображення структури шаблону	6

Короткі теоретичні відомості

Анти-патерни, відомі також як "пастки", представляють собою приклади поширених, але неефективних підходів, які можуть працювати у певних ситуаціях, але загалом призводять до зниження продуктивності та якості коду. Вивчення анти-патернів є важливим для професійного розвитку, оскільки допомагає розробникам уникати типових помилок та виявляти слабкі місця у системах.

Цей термін виник у контексті інформатики, як протилежність до "шаблонів проектування". Вивчення та застосування шаблонів у програмуванні є частиною хорошої практики, а уникнення анти-патернів допомагає розробникам створювати більш якісне програмне забезпечення.

Шаблон Adapter застосовується для узгодження несумісних класів, створюючи проміжний клас, який перетворює інтерфейс одного класу в той, що зрозумілий іншому. Це дає змогу взаємодіяти об'єктам, які інакше не могли б працювати разом, без змін у їхньому початковому коді.

Шаблон Builder використовується для покрокового створення складних об'єктів, ізолюючи процес їх побудови від кінцевого представлення. Це підвищує гнучкість коду та спрощує його підтримку, особливо коли об'єкт має багато параметрів або різні конфігурації.

Command перетворює звичайний виклик методу в клас. Це дозволяє передавати дії як параметри, створюючи черги команд, а також підтримувати скасування та повторне виконання операцій. Основна перевага цього підходу в тому, що він зменшує залежність між відправником і виконавцем дії.

Шаблон Chain of Responsibility організовує обробку запитів за допомогою послідовності обробників. Кожен обробник вирішує, чи обробляти запит, чи передає його наступному в ланцюгу. Це забезпечує гнучкість у зміні послідовності обробки запитів та допомагає реалізувати логіку більш динамічно.

Prototype використовується для створення нових об'єктів шляхом копіювання вже існуючих. Використовується, коли на процес створення об'єкта витрачається багато ресурсів, а копіювання допомагає значно скоротити

витрати. Шаблон Prototype дозволяє зберігати стан об'єкта, це актуально для складних конфігурацій.

Хід роботи:

Тема:

..6 Web-browser (proxy, chain of responsibility, factory method, template method, visitor, p2p)

Веб-браузер повинен мати можливість зробити наступне: мати адресний рядок для введення адреси сайту, переміщатися і відображати структуру html документа, переглядати підключений javascript та css файли, перегляд всіх підключених ресурсів (зображень), коректна обробка відповідей з сервера (коди відповідей HTTP) - переходи при перенаправленнях, відображення сторінок 404 і 502/503.

Враховуючи, що кількість можливих статусів може змінюватися, а логіка обробки кожного з них повинна бути ізольованою, ідеальним рішенням буде використання шаблону **Chain of Responsibility**.

Цей шаблон дозволяє створити ланцюг обробників, де кожен обробник відповідає за певний тип статусу. Якщо обробник не може виконати дію для свого статусу, він передає запит наступному в ланцюзі, доки не знайдеться відповідний обробник. Це забезпечує гнучкість, розширюваність і зменшує залежність між компонентами.

Залежно від коду статусу відповіді (наприклад, 200, 301, 404, 502), потрібно виконувати різні дії: відображати сторінку, перенаправляти на іншу адресу або показувати сторінку з помилкою.

```
3  public abstract class RequestHandler { 8 usages 3 inheritors
4      protected RequestHandler nextHandler; 7 usages
5
6      public void setNextHandler(RequestHandler nextHandler) { 2 usages
7          this.nextHandler = nextHandler;
8      }
9
10     public abstract void handleRequest(Request request); 7 usages 3 implementations
11 }
12
```

Рис. 1. Код абстрактного класу RequestHandler

Абстрактний базовий клас для обробників запитів. Містить посилання на наступний обробник у ланцюгу (nextHandler) і абстрактний метод handleRequest, який реалізується у підкласах.

```

3 public class Request { 1 usage
4     private final int statusCode; 2 usages
5     private final String payload; 2 usages
6
7     public Request(int statusCode, String payload) { no usages
8         this.statusCode = statusCode;
9         this.payload = payload;
10    }
11
12    public int getStatusCode() { no usages
13        return statusCode;
14    }
15
16    public String getPayload() { no usages
17        return payload;
18    }
19 }

```

Рис. 2 – Код класу Request

Клас, який представляє HTTP-запит. Містить інформацію про код статусу (statusCode) і додатковий вміст (payload), що може бути корисним для обробки.

```

-
3 public class SuccessHandler extends RequestHandler { no usages
4     @Override 1 usage
5     @Override 1 usage
6     public void handleRequest(Request request) {
7         if (request.getStatusCode() == 200) {
8             System.out.println("SuccessHandler: Handling successful response.");
9             System.out.println("Payload: " + request.getPayload());
10        } else if (nextHandler != null) {
11            nextHandler.handleRequest(request);
12        }
13    }
14 }

```

Рис. 3 – Код класу SuccessHandler

Конкретний обробник для HTTP-відповідей із кодом статусу 200. Обробляє успішні запити, відображаючи отримані дані (наприклад, рендеринг HTML-сторінки). Якщо код статусу не 200, передає запит наступному обробнику.

```

3 public class RedirectHandler extends RequestHandler { no usages
4     @Override 2 usages
5     @Override 1 usage
6     public void handleRequest(Request request) {
7         if (request.getStatusCode() == 301 || request.getStatusCode() == 302) {
8             System.out.println("RedirectHandler: Handling redirect response.");
9         } else if (nextHandler != null) {
10            nextHandler.handleRequest(request);
11        }
12    }
13 }

```

Рис. 4 – Код класу RedirectHandler

Конкретний обробник для HTTP-відповідей із кодами статусу 301 або 302. Обробляє редиректи (перенаправлення) і виконує відповідну логіку, наприклад, перехід за новим URL. Якщо код статусу не є редиректом, передає запит далі.

```
3      public class ErrorHandler extends RequestHandler { no usages
4          @Override 3 usages
5      f@g public void handleRequest(Request request) {
6          if (request.getStatusCode() == 404) {
7              System.out.println("ErrorHandler: Handling 404 - Page Not Found.");
8          } else if (request.getStatusCode() == 502 || request.getStatusCode() == 503) {
9              System.out.println("ErrorHandler: Handling 502/503 - Server Error.");
10         } else if (nextHandler != null) {
11             nextHandler.handleRequest(request);
12         }
13     }
14 }
```

Рис. 5 – Код класу ErrorHandler

Конкретний обробник для помилок сервера та клієнта, таких як 404, 502, або 503. Відображає відповідне повідомлення про помилку для користувача. Якщо статус не підходить, передає запит наступному обробнику.

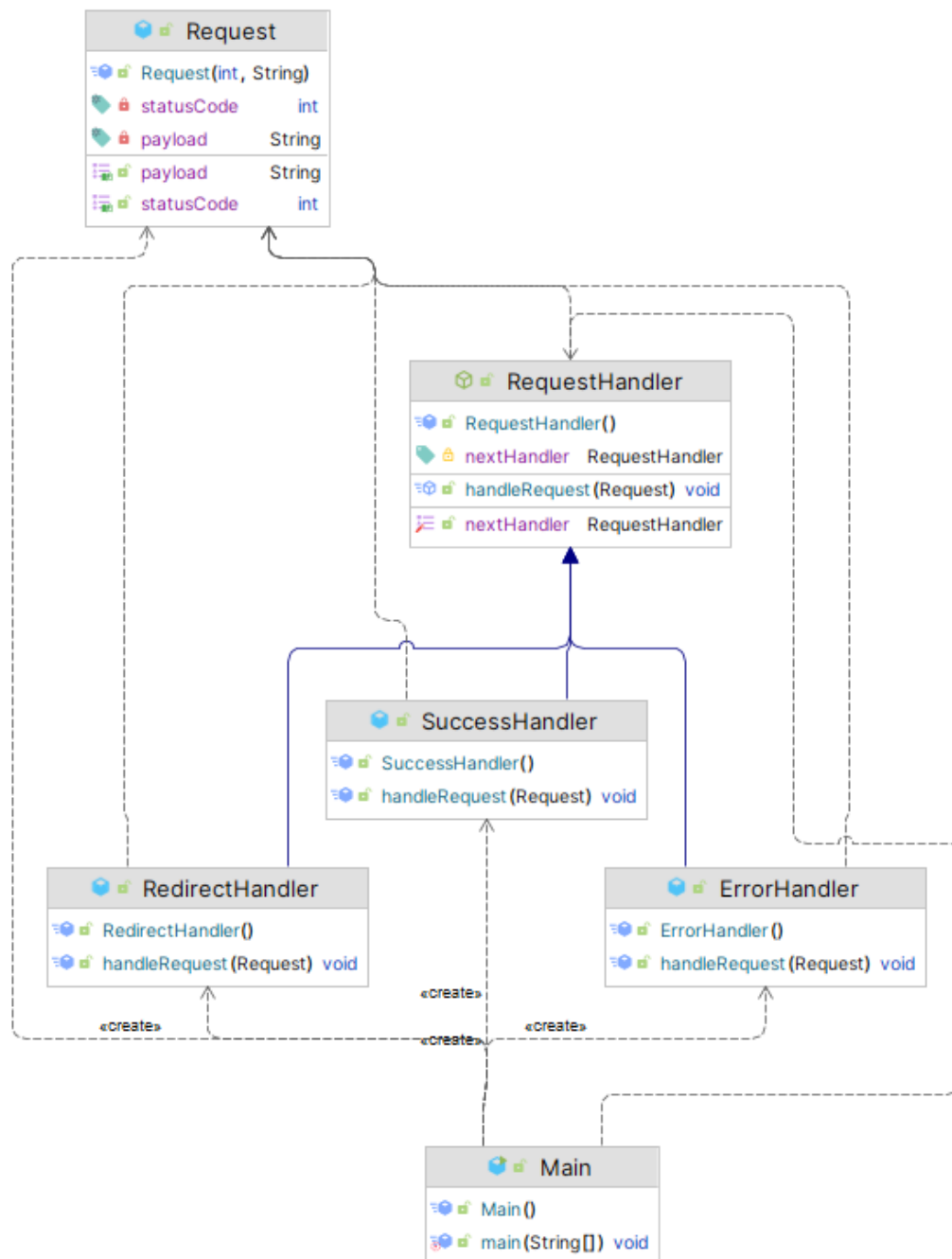


Рис. 6 - Структура шаблону Chain of Responsibility

Шаблон **Chain of Responsibility** є прикладом ефективної розробки, оскільки дозволяє розділити обробку запитів на незалежні модулі, кожен з яких відповідає за конкретний сценарій (наприклад, обробка успішного статусу, редиректу або помилок).

Це забезпечує такі переваги:

1. **Гнучкість:** Ланцюг легко розширити новими обробниками, не змінюючи існуючий код.
2. **Підтримуваність:** Логіка обробки для кожного типу запиту ізольована в окремому класі, що спрощує розуміння та внесення змін.

3. **Зменшення залежностей:** Клієнт, який ініціює запит, не повинен знати, який обробник його виконує. Це сприяє слабкому зв'язуванню компонентів.
4. **Повторне використання коду:** Обробники можна використовувати в інших проєктах або системах, де потрібна подібна обробка запитів.

Цей шаблон демонструє принципи SOLID, особливо принцип єдиної відповідальності та принцип відкритості/закритості, роблячи розробку більш структурованою та ефективною.

Реалізація: https://github.com/Elmir2/trpz_labs

Висновок: У ході виконання даної лабораторної роботи було розглянуто структуру, призначення, переваги та недоліки шаблонів проєктування «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE». Розглянуті патерни програмування мають свої переваги та недоліки, а також використовуються для досягнення різних цілей. Для реалізації частини майбутньої системи було обрано патерн Chain of Responsobility, який найкраще підійде у даному випадку. Далі було реалізовано обраний шаблон, розглянуто його структуру та досліджено його переваги для використання для майбутньої системи.